
In-App Automation Testing Platform

Bridging the Developer–QA Gap with Interactive Test Execution, Live Evidence & Requirement Traceability

Project: ocr-processor (Full-Stack Document Intelligence with Tesseract OCR & Spring AI)
Designed, Architected & Implemented with IBM Bob

101 Automated Cases

Triple-Evidence Model

73.1% Req Coverage

Co-Piloted with IBM Bob

The Core Testing Problem: Status Quo Friction

Disconnected Feedback Loop

Testing happens apart from development in separate pipelines or isolated QA sprints.

- Developers push changes without immediate verification
- Failure reports arrive days later, breaking mental context
- Engineers spend entire cycles context-switching to reproduce
- Significant delay in overall SDLC release velocity

Ambiguous Bug Reports

Bug notices routinely lack the concrete evidence needed for rapid root-cause diagnosis.

- Reports say 'Case Failed' with no runtime parameters
- Missing test data inputs, payloads, and mock configurations
- No screenshots showing what the UI actually rendered
- Heavy back-and-forth: 'Cannot reproduce on my machine'

Late Regression Discovery

New features silently compromise existing functional flows without early warning.

- Unit tests only exercise isolated, mocked layers
- End-to-end user journeys are deferred to manual QA passes
- QA team becomes the first group to discover regressions
- Costly and stressful rework right before production release

The Solution: In-App Automation Testing Platform

Bringing Test Automation Directly to Developers

Rather than pushing test execution off to remote CI queues, ocr-processor embeds the test harness into the running application under the dedicated Tests tab.

✓ Self-Provisioned Execution

Test cases automatically seed isolated database records, mock external APIs, and establish authenticated sessions on demand.

✓ Zero-Friction Selectivity

Developers tick specific test cases or execute complete suites with an asynchronous non-blocking queue.

✓ Triple-Fold Evidence

Every run produces an audit breakdown (Executed / Validated / Observed) paired with real browser screenshots.

✓ Requirements-Grounded Coverage

Direct visual correlation between test runs and Phase 1 functional requirements (REQ-001 through REQ-007).

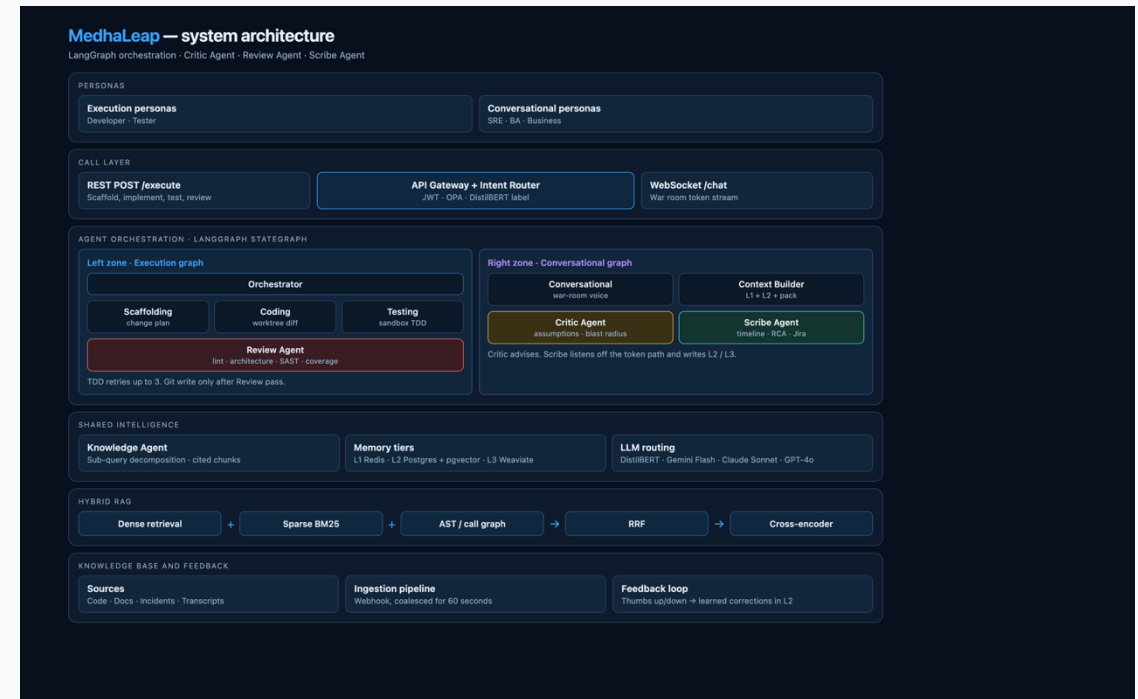


Figure 1: Application Architecture & End-to-End Testing Lifecycle

The Product Under Test: Document Intelligence Pipeline



Multi-Format Ingestion

Upload PDF, PNG, JPG, JPEG, and TIFF documents up to 20MB with real-time UI validation and progress states.



Tesseract OCR Engine

Local optical character recognition with ImageMagick preprocessing, DPI scaling, and TSV confidence filtering.



PostgreSQL TSVECTOR Search

High-speed full-text search indexed via GIN triggers across document names, OCR text, and summaries.



Spring AI LLM Summaries

Automatic document summarisation with smart token truncation (10,000 chars) and one-click retry for failed inferences.



Hardened Security Layer

Stateless authentication via HTTP-only JWT cookies, BCrypt password hashing (factor 12), and automatic 5-attempt account lockout.



Interactive Web Interface

Lightweight Thymeleaf + CSS grid frontend featuring live Search, Create, Profile, and Test Console views.

The Test Console: 101 Test Cases Across 3 Tiers

Unit Tests (32 Cases)

Fast service-layer verification with faked dependencies. Validates JWT parsing & tamper rejection, lockout policies, truncation bounds, and OCR text filters.

Regression Tests (29 Cases)

Integration tests running against a real Testcontainers PostgreSQL instance. Proves full HTTP lifecycles, user isolation, cookie auth, and repository paging.

End-to-End Tests (40 Cases)

Real browser journeys driven by Selenium against the running application, combined with comprehensive API pytest scenarios testing user journeys.

Console Capabilities:

- Selective checkbox picking or full suite execution
- Non-blocking background queue for simultaneous runs
- High-density two-row layout: zero horizontal scroll
- Failed cases automatically bubble to the top of table

☐	#	Case	Expected result	Result	Started	Ended	Duration
☐	30	Short password is accepted FailureSamplesTest.shortPasswordIsAccepted	A password shorter than 8 characters is rejected. Registration requires 8 to 128 characters.	Failed Details	2026-09-26 21:46:04	2026-09-26 21:46:04	5 ms
☐		EXECUTED	VALIDATED	OBSERVED	SCREENSHOT		
		---	---	Expecting actual: 3 to be greater than or equal to: 8			
☐	31	Docx upload is stored FailureSamplesTest.docxUploadIsStored	A docx file is rejected. Uploads accept pdf, png, jpg, jpeg, and tiff.	Failed Details	2026-09-26 21:46:04	2026-09-26 21:46:04	44 ms
☐		EXECUTED	VALIDATED	OBSERVED	SCREENSHOT		
		---	---	Expecting ListN: ["pdf", "png", "jpg", "jpeg", "tiff"] to contain: ["docx"] but could not find the following element(s): ["docx"]			
☐	1	JWT round trip JwtUtilTest.generateAndParseRoundtrip	A generated token parses back to the same user id.	Passed	2026-09-26 21:46:04	2026-09-26 21:46:05	165 ms
☐		EXECUTED	VALIDATED	OBSERVED	SCREENSHOT		
		userid=e9c78235-01e8-42cb-815f-d296915b5c71 username=alice	token issued and parsed	userid=e9c78235-01e8-42cb-815f-d296915b5c71 token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXLTJ1bi9eyj...			
☐	2	Reject a garbage token JwtUtilTest.parseTokenInvalidToken_throwsJwtException	A token that is not a JWT is rejected.	Not run	---	---	---
☐		EXECUTED	VALIDATED	OBSERVED	SCREENSHOT		
		---	---	---	---		
☐	3	Reject a tampered token JwtUtilTest.parseTokenTamperedToken_throwsJwtException	A token whose payload was edited is rejected.	Not run	---	---	---
☐		EXECUTED	VALIDATED	OBSERVED	SCREENSHOT		
		---	---	---	---		
☐	4	Token carries the username JwtUtilTest.claimsContainUsername	The username claim matches the user the token was issued for.	Not run	---	---	---
☐		EXECUTED	VALIDATED	OBSERVED	SCREENSHOT		
		---	---	---	---		

Figure 2: In-App Test Console showing Two-Row Case Layout & Failed-First Sorting

Actionable Evidence & Root-Cause Failure Diagnostics

EXECUTED
Input Parameters

Records user ID, uploaded filename, MIME type, auth headers, and execution flags.

VALIDATED
Assertions & Contracts

Documents required HTTP status, database persistence, and contract preconditions.

OBSERVED
Runtime Outputs

Captures server response JSON, OCR text excerpts, and error messages.

Deliberate Failure Diagnostics (Where · Why · How to Fix)

- Where: pinpoints the exact file and line number in source code (e.g., FailureSamplesTest.java:17)
- Why: contrasts expected contract against observed reality (e.g., Expected length >= 8, but was 3)
- How to Fix: guides the developer on the exact architectural remediation required
- Evidence Card: every non-UI test auto-generates a 640px visual summary card for audits

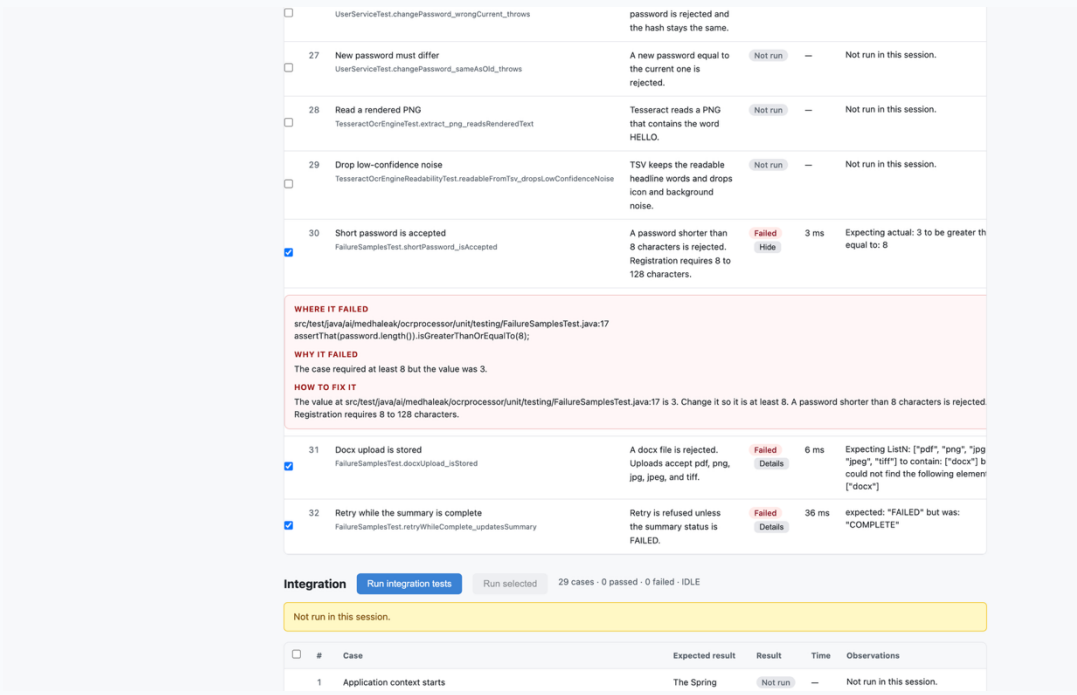


Figure 3: Interactive Failure Explanation Breakdown & Stack Tracing

Requirements-Phase Traceability: Beyond Raw Line Coverage

38 of 52 Criteria Verified (73.1%)

Traditional code coverage tools report lines executed without proving requirements. The Coverage tab maps automated cases directly to acceptance criteria defined in Phase 1:

- REQ-001 Registration (66.7%): Duplicate checks, short passwords, redirects.
- REQ-002 Login (62.5%): Valid auth, generic error messaging, protected routes.
- REQ-003 Change Password (71.4%): Validation, current password verification.
- REQ-004 Home UI (80.0%): Tabs, navigation, and seamless logout flows.
- REQ-005 OCR Search (62.5%): Queries, empty states, and tenant isolation.
- REQ-006 Upload & OCR (87.5%): Ingestion, text extraction, and status transitions.
- REQ-007 LLM Summary (85.7%): Automatic summarisation and failure retries.

Honest Accounting of Open Gaps:
Explicitly explains why remaining criteria (e.g.,
>20MB uploads, live external LLM billing, Tesseract
2-min timeouts) remain open.

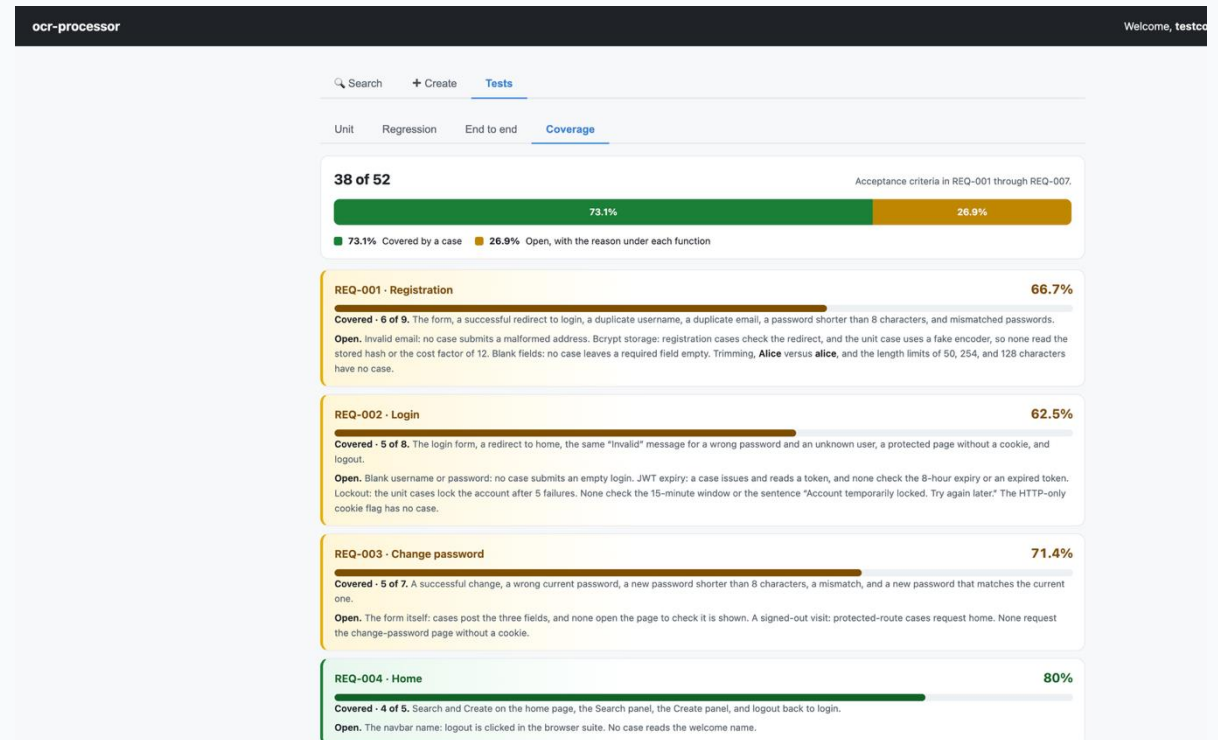


Figure 4: Visual Breakdown of Requirements Coverage & Rationales for Open Items

Co-Piloted with IBM Bob: From Concept to Working System

1

Phase 1: Requirements Engineering

Captured 7 user stories, EARS acceptance criteria, edge cases, and data models in structured documentation.

Key Deliverable: Markdown Spec & ADRs

2

Phase 2: Architecture & Decision Records

Authored 5 Architectural Decision Records (ADRs) evaluating Tesseract, Spring AI, PostgreSQL FTS, and JWT cookies.

Key Deliverable: Markdown Spec & ADRs

3

Phase 3: Design & Technical Spec

Generated end-to-end component wireframes, sequence diagrams, and REST contract schemas before writing code.

Key Deliverable: Markdown Spec & ADRs

4

Phase 4: Full-Stack Implementation

Implemented Spring Boot application, Testcontainers infrastructure, Pytest Selenium runners, and the interactive Test Console.

Key Deliverable: 101 Executable Cases

Measurable SDLC Impact: Accelerating Release Velocity

FEEDBACK LATENCY

Seconds vs. Days

Developers receive immediate failure attribution in their local browser session instead of waiting for nightlies or QA ticket cycles.

REGRESSION CONFIDENCE

Zero Silent Breaks

Executing the 29-case regression suite proves auth, search, upload, and database isolation before committing changes.

DIAGNOSTIC OVERHEAD

Eliminated Guesswork

Every failure includes source line coordinates, contract discrepancy reasons, and visual evidence cards.

HANDOFF QUALITY

Clean QA Drops

Testing teams receive verified builds where baseline functionality is proven, freeing QA for exploratory testing.

Summary: Testing as an Active Development Tool

- Complete Transparency: Every test is self-documenting with Executed, Validated, and Observed evidence.
- Integrated Developer Experience: Run Unit, Regression, and End-to-End suites without leaving the application.
- Requirements-Driven Quality: Continuous tracking against formal acceptance criteria (REQ-001..REQ-007).
- Accelerated SDLC: Catch defects earlier, eliminate handoff friction, and deliver faster with confidence.

Thank you! Ready for Live Demo & Q&A